EcoPulse

TECHNICAL DOCUMENTATION

Backend work: architecture, data flow and models

How a citizen's photo becomes a ranked, de-duplicated, geo-located incident on the city's screen in about a second. Covers the services, the database, the background pipeline, the scoring models and how realtime updates reach every client.

API FastAPI (Python 3.12), async, modular monolith	**Database** PostgreSQL 16 + PostGIS 3, SQLAlchemy 2 async, Alembic	**Messaging** Redis: pub/sub events, job queue, rate limits	**Geo & ML** PostGIS, H3, Shapely, scikit-learn DBSCAN, NumPy
AI Claude vision (optional) or local heuristic classifier	**Media** Pillow re-encode, dHash, local FS or S3/MinIO	**Open data** Open-Meteo (AQI, weather, elevation), OSM Overpass	**Clients** React + TS: citizen app (Capacitor), command center

1. System architecture
2. Code layout and modules
3. Data model
4. End-to-end data flow of a report
5. Report pipeline and duplicate detection
6. AI classification
7. Risk model and priority
8. Incident lifecycle and verification
9. Background worker and jobs
10. Hotspots, H3 grid and flood model
11. Area (Pulse) scores and recommendations
12. Realtime delivery (Redis + WebSockets)
13. External data providers
14. Security and privacy
15. API surface
16. Running, config and testing

Source of truth: the `backend/` folder. Every formula below comes from the module docstrings and code it cites, e.g. `app/intelligence/risk.py`.

1 · SYSTEM ARCHITECTURE

One deployable backend, two front-ends, three stores

The backend is a **modular monolith**: one FastAPI application with strict module boundaries (`app/modules/*`). A **background worker** does the slow work (classification, de-duplication, analytics) so the HTTP request a citizen makes returns fast. In development the worker runs *inside* the API process; in production it can run separately as `python -m app.worker`.

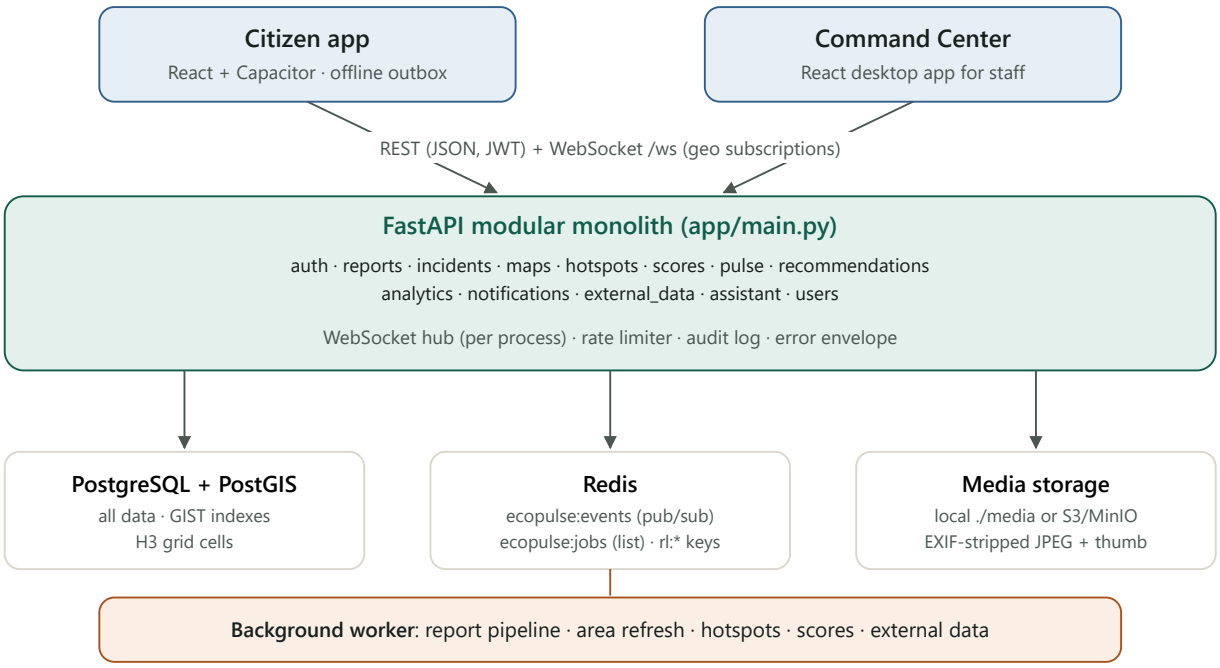


Figure 1. Runtime components. External providers (Open-Meteo, Overpass, Anthropic) are called from the worker / assistant only.

Graceful degradation: every dependency except Postgres is optional

Missing	What the backend does instead
Redis	In-process `asyncio.Queue` for jobs, direct hub dispatch for events, in-memory fixed-window rate limiting (single process only).
Anthropic API key	Local `HeuristicClassifier` (keyword lexicon + image colour stats, confidence capped) and a deterministic intent router for the assistant.
Internet / providers down	Clearly labelled demo AQI/weather generators (`is_demo = true`); Overpass results are cached 30 days.
S3	Local filesystem under `backend/media`, served at `/media`.

2 · CODE LAYOUT

Where things live

```
backend/
├─ app/main.py            FastAPI app, middleware, /health, /meta, /ws, lifespan (starts worker + bus
listener)
├─ app/worker.py          job consumer + periodic ticker + debounced AreaRefresher
├─ app/models.py          SQLAlchemy ORM (all tables, PostGIS geometry columns)
├─ app/core/              config · db · security (JWT, bcrypt, RBAC) · geo (H3, privacy snap)
│                         storage (image processing, dHash) · ratelimit · audit · errors · catalog
├─ app/modules/<name>/    router.py (HTTP endpoints) + service.py (domain logic) per module
│    reports/pipeline.py   report → incident pipeline (classification, dedupe, scoring)
├─ app/intelligence/      risk.py · hotspots.py · grid.py · scores.py · recommendations.py
├─ app/ai/                classifier.py (Claude vision / heuristic) · text.py (similarity)
├─ app/providers/         open_meteo.py · overpass.py · satellite.py · service.py (refresh + demo
fallback)
├─ app/realtime/          bus.py (Redis pub/sub) · hub.py (WebSocket fan-out) · jobs.py (queue)
├─ app/seed/              demo city: Bengaluru neighbourhoods, synthetic reports through the real pipeline
├─ alembic/               schema migrations (creates the postgis extension)
└─ tests/                 unit + integration tests, e2e realtime test
```

3 · DATA MODEL

PostgreSQL + PostGIS (SRID 4326)

All geometry is WGS84 lon/lat. Spatial columns have **GIST indexes**; distance filters use `ST_DWithin` on `geography` casts, so radii are in real metres.

Group	Tables	Notes
Identity	`users`, `refresh_tokens`, `audit_logs`, `points_ledger`	Roles: citizen, moderator, analyst, officer, admin. Refresh tokens stored as SHA-256 hashes.
Citizen input	`reports`, `media`, `verifications`	A report is *one person's observation*: exact point, accuracy, H3 cell, AI output, duplicate score + signals, `client_id` (idempotency).
Operational	`incidents`, `incident_events`, `assignments`, `resolution_updates`	An incident is *the real-world problem*; many reports merge into one. Stores `geom` (exact) **and** `public_geom` (snapped). Events are an immutable timeline.
Geography	`locations`, `grid_cells`, `green_zones`, `sensitive_sites`, `waste_points`	Neighbourhood polygons, H3 r8 analytic grid, OSM-derived parks / schools / hospitals / waste points.
Analytics	`hotspots`, `environmental_scores`, `recommendations`, `interventions`	Hotspots are polygons (observed or predicted). Scores keep full history with explanations.
Other	`notifications`, `external_data`, `categories`	Notifications deduplicated by a unique `(user_id, dedupe_key)`. External data tagged with source and `is_demo`.

Report vs incident. This split is the core design choice. Ten people photographing the same garbage pile produce ten `reports` but one `incident`, whose `reporter_count = 10` raises its confidence and risk instead of flooding the staff queue with duplicates.

4 · END-TO-END DATA FLOW

From "Send" to every screen in about a second

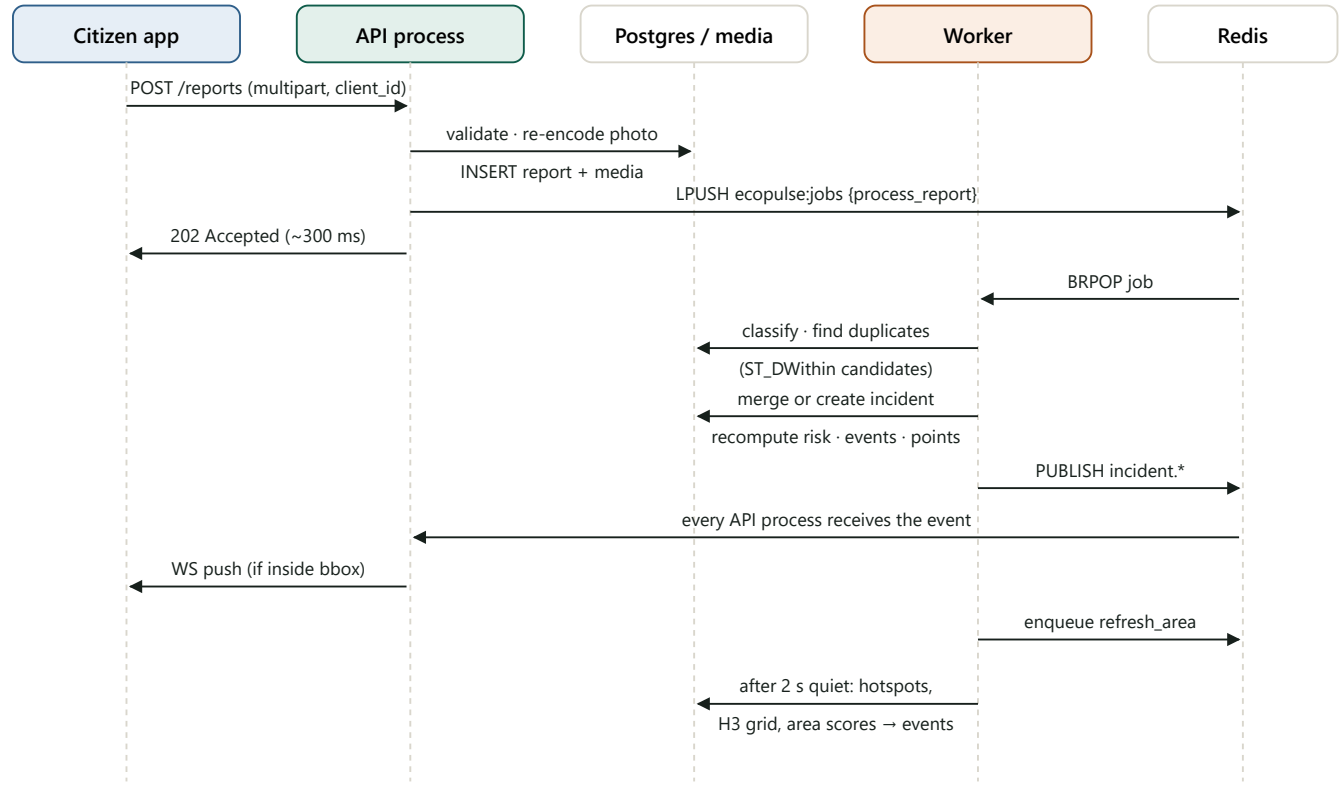


Figure 2. Sequence of one report. Only the first four arrows are on the citizen's request path; everything else is asynchronous.

Synchronous part request path

Validate category / coordinates / sizes → idempotency check on `(user_id, client_id)` → rate limit (30 reports/hour) → photo required unless category is noise / air / flooding / waterlogging / other → re-encode image, compute dHash, store original + 360 px thumb → insert rows → audit entry → enqueue job → emit `report.received` (coordinates rounded to 2 decimals) → **202**.

Asynchronous part worker

`process_report`: classification → duplicate detection → merge into or create incident → aggregate update → lifecycle events → EcoPoints → risk + priority → notifications → realtime broadcast → debounced area refresh. Each stage is described on the next pages.

5 · REPORT PIPELINE

process_report, step by step

File: `app/modules/reports/pipeline.py`

1. **Classify.** Load the evidence photo from storage, call `classify_report(category, description, jpeg)`. Store AI category, label, confidence, severity (1 to 4) and provider. For noise, a measured dB level overrides the heuristic: ≥85 dB gives severity 4, ≥75 gives 3, ≥65 gives 2.
2. **Locate.** `ST_Intersects` with neighbourhood polygons; if outside all, attach to the nearest within 3 km.
3. **Find duplicate candidates.** SQL pre-filter: same *domain*, not terminal, last report inside the category's time window, within a radius in metres via `ST_DWithin(geography)`.
4. **Score each candidate** on five signals (table below). Best score ≥ **0.62** means merge.
5. **Merge**: link report, re-aggregate the incident (report count, distinct reporters, centroid of all reports via `ST_Centroid(ST_Collect)`, max severity, mean AI confidence), add `report_merged` event. If 3+ independent reporters, then `auto_verified`. **Or create**: new incident with ref `EP-XXXXXX`, title, events `created` → `ai_classified`.
6. **Risk + priority** recomputed (section 7). If the incident just became verified, add `prioritized` event, award points and notify every reporter.
7. **Notify.** "Your report joined …" for merges; a severity-4 new incident triggers `notify_nearby` to residents within 1.5 km.
8. **Broadcast** `incident.created` / `incident.updated` and enqueue `refresh_area` for the incident's location.

Duplicate-detection score

Signal	Weight	Value in [0, 1]
spatial	0.35	1 − distance / radius. Radius = clamp(2 × GPS accuracy + 80, 60, 250) m; ×1.6 (max 400 m) for diffuse domains: air, noise, flood.
category	0.25	1 if same category, 0.6 if same domain, else 0
temporal	0.15	1 − Δt / window. Window = category persistence, clamped to 2 to 14 days.
image	0.15	1 − Hamming($dHash_{new}$, $dHash_{closest}$) / 32, against up to 25 existing photos
text	0.10	Max over existing descriptions of token Jaccard + trigram cosine

Weights are **renormalised over the signals actually present**: a report without photo or text is judged on spatial / category / temporal alone, rather than being penalised with zeros. The score and per-signal breakdown are saved on the report (`duplicate_score`, `duplicate_signals`) so staff can see why two reports were merged.

6 · AI CLASSIFICATION

Two interchangeable classifiers

File: `app/ai/classifier.py`

	Claude vision (`ClaudeVisionClassifier`)	Local heuristic (`HeuristicClassifier`)
When	`ECOPULSE_ANTHROPIC_API_KEY` set	Always available; fallback
Input	EXIF-stripped JPEG + description	Description keywords + colour statistics of a 96 px HSV thumbnail
Output	Schema-constrained JSON: category, subtype, severity, confidence, rationale	Same `Classification` dataclass. Features: green, brown, grey haze, water blue, dark, clutter. Confidence is **capped**.

The classifier **never overrides the citizen's category**. Its answer is stored alongside it with `agrees_with_user`, used to estimate severity and flag mismatches for moderators. The same classifier powers `POST /ai/classify`, which the app calls for the instant "Looks like: Garbage" preview; nothing is stored by that call.

7 · RISK MODEL (`RISK-1`)

Why an incident ranks where it does

File: `app/intelligence/risk.py`, inputs gathered in `incidents/service.py · recompute_risk`

$$\text{Risk} = 100 \cdot \Sigma\, w_i \cdot c_i \qquad \text{every component } c_i \in [0, 1]$$

Component	w	Definition
severity	0.28	severity/4 × (0.7 + 0.3 · min(hazard_weight/1.6, 1))
frequency	0.16	$1 - e^{-\text{reporters}/5}$
exposure	0.16	$1 - e^{-\text{people within 500 m} / 8000}$; population from the H3 cell (or neighbourhood density)
persistence	0.10	days open / typical persistence of the category
sensitivity	0.12	Schools, hospitals, kindergartens, care homes, clinics within 300 m (distance-weighted) + area sensitivity
trend	0.08	reports in the last 72 h / 5
external	0.10	Air: latest US AQI / 300. Flood: (rain next 48 h + ½ past 24 h) / 60 mm.

If no external signal exists, its weight is **redistributed** proportionally instead of counting as zero. Resolved incidents keep 25% of their score (visible, not competing). The full explanation (inputs, components, weights, contributions, top three drivers) is stored in `incidents.risk_components`; the apps turn the top drivers into sentences like "many people live nearby and a school is close by".

Risk	≥ 80	60 to 79	35 to 59	< 35
Priority	P1 · Urgent	P2 · High	P3 · Medium	P4 · Low

Because age and weather change over time, the worker **re-scores every open incident** on each periodic regeneration.

8 · INCIDENT LIFECYCLE

State machine and verification

```
reported → ai_classified → verified → prioritized → assigned → action_started → resolved →
citizen_verified
                                                                                  │
                        (3+ "still there" votes after resolution) ←── reopened ──┘        dismissed
(moderator)
```

Every transition goes through `add_event()`, which writes an immutable `incident_events` row (from/to status, actor, note, JSON data) and updates the status. The timeline shown in both apps is just this table.

Transition	Trigger
→ **verified**	Automatically when 3+ independent reporters merge; or community vote: **yes ≥ 3 and yes > 2 × no**; or a moderator.
→ **prioritized**	Immediately after verification, with the freshly computed priority.
assigned → **resolved**	Officer actions (permission `incident.assign` / `incident.resolve`); resolution can attach an after-photo.
→ **citizen_verified**	Resolution-phase vote: 3+ say it's gone and they outnumber "still there" 2:1.
→ **reopened**	Resolution-phase vote: 3+ say it's still there (2:1), so `resolved_at` is cleared, risk is recomputed and reporters are notified.

Confidence (0 to 1) blends AI confidence, corroboration ($1 - 0.6^{\text{reporters}}$) and a Laplace-smoothed community vote share, with the vote weight rising to 35% as votes accumulate.

9 · BACKGROUND WORKER

Two loops: consumer and ticker

Files: `app/worker.py`, `app/realtime/jobs.py`

Consumer

Blocks on `BRPOP ecopulse:jobs` (or the in-process queue). Jobs are tiny JSON documents `{"name", "args"}`:

- `process_report`: the pipeline above
- `refresh_area`: marks a location dirty
- `regenerate`: full analytics rebuild
- `refresh_external`: pull AQI / weather / OSM

Admins can trigger the last two through `POST /admin/jobs/…`.

Ticker (every second)

- **AreaRefresher** debounce: waits for **2 s of quiet**, then recomputes once for all dirty locations. Hotspots + grid run at most every 10 s. A burst of 50 reports causes *one* recompute.
- Every **5 min** (`hotspot_interval_seconds`): hotspots, grid, re-score open risk, area scores, recommendations.
- Every **30 min** (`external_interval_seconds`): refresh external data, then check for flood alerts.

Hotspots, the H3 grid and the flood model

Observed hotspots (`hotspot-1` , `app/intelligence/hotspots.py`)

- Per domain, incidents from the last **45 days** are clustered with **weighted DBSCAN** (scikit-learn, haversine metric, **eps = 350 m**).
- Incident weight = severity/2 × √reporters (resolved ×0.3, since they still show recurrence). A cluster needs total weight ≥ 4 and ≥ 2 incidents (or one incident reported 12+ times).
- Hotspot risk = 100 × (0.45·top-risk + 0.25·density + 0.15·trend + 0.15·persistence), where top-risk is the mean of the five riskiest open incidents, density = $1 - e^{-\text{open}/8}$, trend is 14-day report growth vs the previous 14 days, and persistence = days since first report / 30.
- **Stable identity**: a new cluster inherits the id of the existing hotspot it overlaps most, so clients animate updates. Vanished clusters become *cooling*, then *resolved* after 14 days.

Predicted hotspots

For each H3 cell and domain, weekly report counts over 8 weeks are fitted with a least-squares line. A rising slope with a 14-day forecast ≥ 4 reports, where there's no observed hotspot, gives a **predicted** hotspot, with confidence proportional to the supporting history and always labelled as a forecast.

H3 analytic grid (`grid-1` , resolution 8, about 0.74 km² per cell)

Built once by polyfilling the union of neighbourhood polygons; refreshed after every area change. It's the common unit for heatmaps, population exposure and the flood model. Cell risk = root-sum-square of the open incidents' risks (one 80 gives 80; two 60s give 85), capped at 100.

Flood model (`flood-1` , `app/intelligence/grid.py`)

```
observed  = 1 − e^(−w/2)      w = severity-weighted flooding/waterlogging reports in the last 72 h
predicted = (0.25 + 0.75·rain) · (0.30·low_lying + 0.35·history + 0.20·drainage + 0.15·impervious)

rain       = (forecast next 48 h + ½ · past 24 h) / 80 mm
low_lying  = (city median elevation − cell elevation) / 25 m        (Open-Meteo elevation)
history    = 1 − e^(−flood/waterlogging incidents ever in the cell / 3)
drainage   = 1 − e^(−open drainage incidents / 2)
impervious = 1 − green fraction
```

Observed and predicted are separate columns and never mixed. After each external refresh, if a neighbourhood's peak predicted risk is ≥ 0.45 **and** rose by ≥ 0.15, residents get one "Flood risk increased" alert per area per day (deduplicated by key).

Environmental Pulse score (`pulse-1`)

File: `app/intelligence/scores.py` . Higher is better, 0 to 100, reproducible from stored data.

Load for a domain = Σ over open incidents of severity × hazard × √reporters, per km². Domain score = $100 \cdot e^{-\text{load}/k}$.

Dimension	Inputs	k	Weight
air	60% external AQI (100 − AQI/3) + 40% citizen air load	6	0.22
water	citizen water load (pollution, sewage)	5	0.18
waste	citizen waste load (garbage, dumping, plastic, e-waste)	8	0.20
noise	citizen noise load − 0.8 pt per dB measured average above 55	5	0.12
green	green-area share (25% share gives 100) − 4 pt per tree-loss incident per km²	none	0.15
clean	60% share of last-60-day incidents resolved + 40% speed ($e^{-\text{median days open}/14}$)	none	0.13

Each score row stores its inputs, method string and weights, which is exactly what the app's "How do we know this?" sheet displays. If a location's score drops by ≥ 5 points between recomputes, an alert is emitted. Grades: good ≥ 75, fair ≥ 55, poor ≥ 35, otherwise critical.

Recommendation engine (`rec-1`)

Every recommendation separates **observed** facts, model **inference** (with model version) and ranked **actions**, each tagged with the rule that produced it.

1. **Hotspot playbooks** for persistent waste / water / air / noise hotspots.
2. **Waste-point siting**: greedy maximal-coverage choice of new collection points for open waste incidents more than 250 m from any known point.
3. **Green corridors**: short links between nearby green zones, scored by how little green they cross, population along them and air risk.
4. **Flood preparedness** for locations with elevated `flood-1` risk.
5. **Silence zones**: noise incidents within 300 m of schools and hospitals.

Upserted by a stable key, so regenerating never duplicates and staff accept/dismiss decisions survive. Accepted recommendations become tracked `interventions` .

Grounded assistant

With an API key, Claude answers through a **tool-use loop** over EcoPulse data tools (`app/modules/assistant/tools.py`) and cannot see anything the tools do not return. Without a key, a deterministic intent router calls the same tools. Every answer returns the tool calls it made. Rate limited.

12 · REALTIME DELIVERY

Redis pub/sub → per-process hub → filtered WebSockets

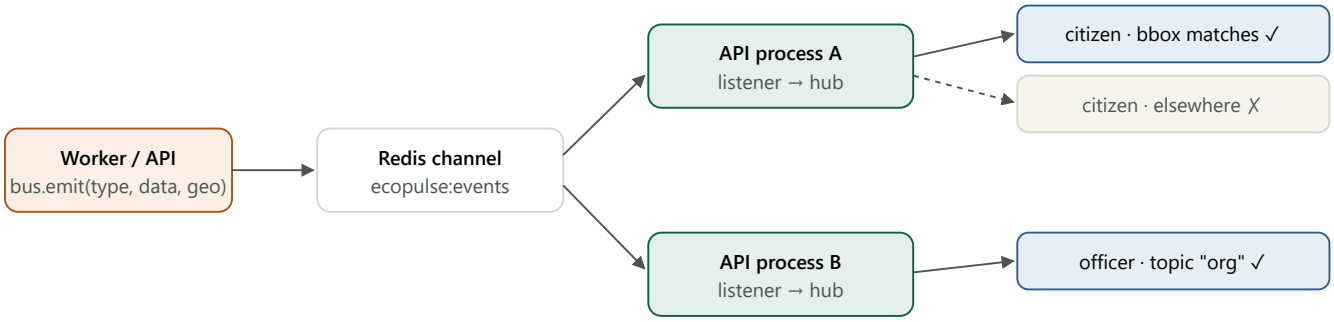


Figure 3. Fan-out. Every API replica receives every event and filters per connected client, so the system scales horizontally.

Protocol (`WS /ws?token=…`)

```
client → {"type":"subscribe","topics":["incidents","hotspots","scores","alerts","pulse"],"bbox":[w,s,e,n]}
client → {"type":"ping"}                                    server → {"type":"pong"}
server → {"id","type":"incident.updated","topic":"incidents","ts","geo":{"lon","lat"},"data":{…}}

event types: incident.created | incident.updated | incident.resolved | report.received
             hotspot.created | hotspot.updated | hotspot.cooled | environmental_score.updated
             alert.created | notification.created (targeted by user_ids)
```

Filtering rules (`app/realtime/hub.py · Client.wants`)

- Events with `user_ids` (notifications) go only to those users.
- Staff can subscribe to `org` and receive everything; citizens are limited to public topics.
- Geo-located events are delivered only if inside the client's viewport bbox (padded 25% each side), unless the client follows the city-wide `pulse` topic.
- **Back-pressure**: each client has a 500-event queue. If it fills, it is cleared and replaced with a single `resync` message, so one slow phone never blocks others.
- Idle sockets close after 90 s without a message; clients send heartbeats.

Client side (`packages/core/src/realtime.ts`): auto-reconnect with exponential backoff + jitter, heartbeat, and subscription replay. When the map moves, the new bbox is re-sent. Events patch the React Query cache in place, so the map, queue, feed and stats update without refetching.

13 · EXTERNAL DATA

Open data providers

Provider	Data	Used for
Open-Meteo	US AQI, PM2.5, PM10 per neighbourhood centroid; rainfall past 24 h + forecast 48 h; elevation	Air score, risk external signal, flood model
OSM Overpass	Parks / green areas, schools, hospitals, clinics, waste points	Green score, sensitivity in risk, waste-point siting, silence zones
Satellite NDVI	Not configured (needs imagery credentials)	Nothing is fabricated in its place

No keys needed. Results go to `external_data` with source name and timestamp. On failure, deterministic demo generators are used and flagged `is_demo = true`, which every UI shows as "Sample data". Overpass responses are cached 30 days. When live geodata becomes available it upgrades demo geodata and triggers a full regenerate.

14 · SECURITY AND PRIVACY

Defence in depth

Authentication

- Passwords: **bcrypt**, 12 rounds.
- **JWT access tokens** (HS256, 30 min) carrying user id and role.
- **Rotating refresh tokens** (30 days), stored only as SHA-256 hashes. Each refresh revokes the old token. **Reuse of a revoked token** is treated as theft: every token for that user is revoked and the event is audited.
- Guest accounts are real users, so upgrading keeps reports and points.
- Production refuses to boot with the dev JWT secret or one shorter than 32 characters.

Authorisation (RBAC)

Endpoints depend on **permissions**, never role names: e.g. `incident.view_exact` (all staff), `incident.assign` / `resolve` (officer, admin), `recommendation.manage`, `users.manage` and `audit.view` (admin), `jobs.run`.

Location privacy

- Exact coordinates are visible only to the reporter and staff.
- Public APIs and all realtime events use `public_geom`, a **deterministic ~120 m grid snap** (`core/geo.py · snap_public`). The same input always maps to the same output, so repeated queries can't be averaged to recover the true point.
- `report.received` events round coordinates to 2 decimals (about 1 km).
- Anonymous reports hide the reporter from staff too.
- Noise is measured on the phone; only the dB number is uploaded.

Input and abuse protection

- Pydantic validation on every field (ranges, lengths).
- Images: type allow-list, 12 MB limit, `verify()` decode check, pixel bound, **re-encoded** (EXIF/GPS stripped), max edge 1920 px.
- Fixed-window **rate limits** (Redis): auth, reports (30/h), verification, classification (40/h), assistant.
- **Audit log** for every state change, with actor and IP.
- Security headers (`nosniff`, `DENY` framing, referrer policy); CORS allow-list.
- Uniform error envelope `{"error": {"code", "message", "details"}}`.

15 · API SURFACE

Main endpoints

Full interactive OpenAPI docs: `http://localhost:8000/docs`

```
POST /auth/guest | /auth/register | /auth/login | /auth/refresh | /auth/logout
GET|PATCH /auth/me      GET /auth/me/points
POST /reports  (202)        POST /ai/classify (preview, nothing stored)
GET  /reports?mine=true    GET /reports/{id}
GET  /incidents  /incidents/{id}                          POST /incidents/{id}/verify  (yes|no|unsure)
POST /incidents/{id}/assign | /start | /resolve (after-photo) | /moderate             (staff)
GET  /map/layers | /map/incidents (zoom-adaptive clustering) | /map/heatmap | /map/grid | /map/locations
     /map/green-zones | /map/waste-points | /map/sensitive-sites | /map/aqi | /map/recommendations | /map/search
GET  /hotspots  /hotspots/{id}
GET  /environmental-score?lat&lon | ?location_id      /environmental-score/locations   /{id}/history
GET  /pulse     GET /recommendations    POST /recommendations/{id}/decision    GET|PATCH /interventions
GET  /analytics/overview   GET /notifications   POST /notifications/{id}/read | /read-all
POST /assistant/ask         GET /external-data/status | /external-data/location/{id}
GET  /users  /users/assignable     PATCH /users/{id}   GET /audit-logs
POST /admin/jobs/{regenerate|refresh_external}
GET  /health  /meta         WS /ws
```

16 · RUNNING, CONFIGURATION AND TESTS

Operating it

```
docker compose up -d db redis              # PostGIS + Redis
cd backend && pip install -r requirements.txt && cp .env.example .env
alembic upgrade head                       # schema + postgis extension
python -m app.seed.run                     # demo Bengaluru city (~2 min, real pipeline, OSM + Open-Meteo)
uvicorn app.main:app --reload --port 8000  # API + embedded worker
python -m app.worker                       # (production) separate worker process
```

Setting (env, prefix **ECOPULSE_**)	Default	Effect
DATABASE_URL	postgres@localhost/ecopulse	asyncpg connection
REDIS_URL	redis://localhost:6379/0	Bus, queue, rate limits; unset gives in-process fallbacks
ANTHROPIC_API_KEY	none	Enables Claude vision + tool-use assistant
STORAGE_BACKEND	local	`s3` + S3_* vars for MinIO/S3 with presigned URLs
EMBEDDED_WORKER	true	Run the worker inside the API process
PUBLIC_LOCATION_PRECISION_M	120	Privacy snap grid size
HOTSPOT / EXTERNAL_INTERVAL_SECONDS	300 / 1800	Periodic regeneration / data refresh
EXTERNAL_DATA_ENABLED	true	`false` forces labelled demo providers

Testing

`cd backend && pytest` runs unit tests for the pure models (duplicate score, risk, privacy snap, scores) and integration tests against the seeded database. `python tests/e2e_realtime.py` against a running API exercises the whole loop: submit a report, wait for the pipeline, and assert the WebSocket event arrives.

Data honesty. Neighbourhood names and centres are real; boundaries are synthetic (Voronoi) and population figures are demo estimates. Both are flagged in the API and UI. Seeded incidents are synthetic (`is_demo = true`) but are processed by the real pipeline.